Object Detection & Localization

Week 14 — Session 1: From “what” to “where” — boxes, IoU, and NMS

Practical Image Processing

Lecture 14.1 ▪ 60–70 min

The big idea

- Week 13: a CNN answers *what* is in an image (a digit, a class).
- This week: answer *where* it is — draw a box around each object.
- "What + where" together is **object detection**: the workhorse of modern vision.
- **Today (Session 1):** boxes, sliding windows, and the two tools every detector needs — **IoU** and **NMS**.

Where we are heading

DocScan must first *find* the meter — the document, then the digit strip — before Week 13's reader can read it. Detection is the missing "find it" step.

Roadmap — Session 1

Three tasks: classify, localize, detect

The sliding-window idea

IoU: measuring overlap

NMS: keeping the best box

In-class exercise

Wrap-up

Three tasks: classify, localize, detect

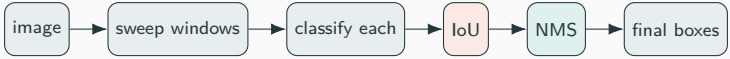
Classification vs localization vs detection

Task	Output	Example
Classification	one label for the whole image	"this crop is a 7"
Localization	one label + *one box*	"a meter, here"
Detection	*many* labels + boxes	"meter here, dial there"

Key idea

Detection = classification you have run everywhere, then cleaned up into a few good boxes.

The detection pipeline at a glance

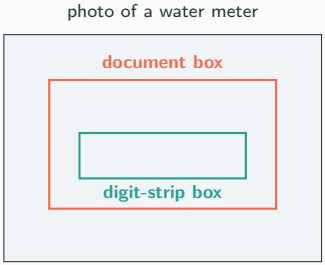


- Today we build the whole chain by hand; Session 2 replaces "sweep + classify" with one CNN.
- **IoU** and **NMS** (the last two boxes) never change — they survive into every modern detector.

Key idea

Keep this chain in view: everything today is one box on this diagram.

Our running problem, upgraded



- Week 13 read an *already-cropped* 8×8 digit.
- Reality: the phone photo has the meter *somewhere* in the frame.
- DocScan must localize the **document**, then the **digit strip**, *then* read.

Key idea

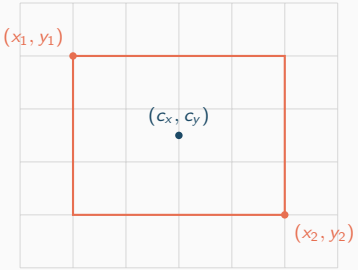
Every idea today plugs into "find the strip before you read it."

A bounding box is just four numbers

- Corner form: (x_1, y_1, x_2, y_2) — top-left and bottom-right.
- Center form: (c_x, c_y, w, h) — what detectors predict.
- Convert freely: $w = x_2 - x_1$, $c_x = x_1 + w/2$.

Area of a box

$$\text{area} = (x_2 - x_1)\,(y_2 - y_1)$$



Localization = regression (one object)

- Take Week 13's CNN backbone; it already turns an image into features.
- Add *two heads* on top of those features:
  - a **classification head** $\rightarrow$ class scores (softmax, as before);
  - a **box-regression head** $\rightarrow$ 4 numbers (c_x, c_y, w, h).
- Train the box head to minimize a regression loss vs. the true box.

Box loss (smooth L1 / MSE)

$$L_{\text{box}} = \sum_{k \in \{c_x, c_y, w, h\}} \left(\hat{k} - k\right)^2$$

Key idea

Classification predicts a category; regression predicts numbers. A box is just 4 numbers.

Box loss, with real numbers

Predicted centre-form box $\hat{b} = (30, 20, 100, 40)$, ground truth $b = (32, 18, 96, 44)$.

Squared error, term by term

$$(30-32)^2 + (20-18)^2 + (100-96)^2 + (40-44)^2$$
$$= 4 + 4 + 16 + 16 = \mathbf{40}$$

- The width/height terms dominate here — the box is the right place but a little too wide/short.
- Gradient descent nudges $(\hat{c}_x, \hat{c}_y, \hat{w}, \hat{h})$ to shrink this number, exactly like Week 13.
- In practice we use *smooth L1*: squared for small errors, linear for large ones — robust to outliers.

But real images have *many* objects

- A single box head can localize *one* object per image.
- A meter photo may show the body, several dials, a serial plate…
- We do not know *how many* boxes to output in advance.

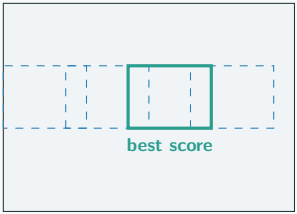
✗ The core difficulty of detection

Classification has a fixed-size output (10 digit scores). Detection has a *variable* number of outputs — that is what makes it hard.

The sliding-window idea

Sliding windows: brute force, but it works

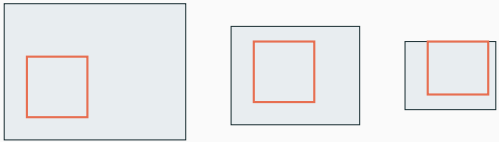
1. Pick a window size.
2. Slide it over the image (like a Week-3 kernel, but bigger).
3. Run the **classifier** on each crop: "digit strip here? yes/no + score."
4. Keep every window that scores high.



the window that lands on the object scores highest

Objects come in many sizes: the image pyramid

- One window size only finds objects of *that* size.
- Fix: shrink the image repeatedly (a **pyramid**) and slide the *same* window on each level.
- A big object at full size looks small a few levels up — now the window fits.



Same coral window, three scales: scale-invariant search for free.

The bill: why brute force hurts

Rough window count

$$\text{windows} \approx \underbrace{\frac{W\,H}{s^2}}_{\text{positions}} \times \underbrace{L}_{\text{scales}}$$

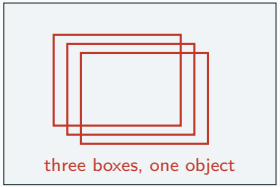
- A 640×480 image, stride s=8, L=5 scales $\Rightarrow \approx$ 24,000 classifier calls *per image*.
- Add a few aspect ratios and it explodes further.
- Slow — but conceptually it *is* detection. Session 2 makes it fast.

Good practice

Sliding window is the mental model. Every modern detector is an *efficient* version of this same sweep.

Now the mess: overlapping detections

- A good detector fires on *several* nearby windows for one object.
- You wanted 1 box; you got 5 overlapping boxes.
- We need to (a) measure "how much do two boxes overlap" and (b) collapse duplicates.



Key idea

The fix is two ideas: **IoU** (measure overlap) and **NMS** (keep the best, drop the rest).

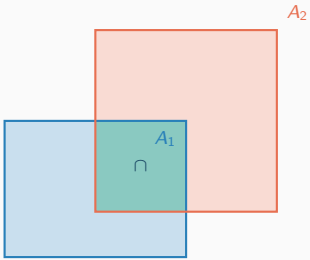
IoU: measuring overlap

Intersection over Union (IoU)

- **IoU** $= \dfrac{\text{area of overlap}}{\text{area of union}}$.
- 0 = no overlap, 1 = identical boxes.
- It is how we score a predicted box against the truth, *and* how NMS spots duplicates.

Union without double-counting

union $= A_1 + A_2 -$ intersection



IoU, by hand

Boxes in (x_1, y_1, x_2, y_2): $A = (1, 1, 5, 4)$, $B = (3, 2, 7, 6)$.

Overlap rectangle

x: $[\max(1,3), \min(5,7)] = [3,5] \Rightarrow 2$

y: $[\max(1,2), \min(4,6)] = [2,4] \Rightarrow 2$

inter $= 2 \times 2 = \mathbf{4}$

Areas and IoU

$A_1 = (5-1)(4-1) = 12$

$A_2 = (7-3)(6-2) = 16$

union $= 12 + 16 - 4 = 24$

$\text{IoU} = \dfrac{4}{24} = \mathbf{0.167}$

Overlap width/height clamp at 0: if the ranges do not cross, intersection is 0, not negative.

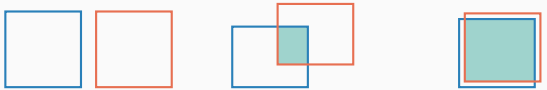
Reading IoU numbers

IoU	Meaning
0	boxes do not touch
0.1–0.3	slight overlap; probably different objects
0.5	the classic “a hit” threshold for evaluation
0.7–0.9	tight, well-aligned prediction
1.0	pixel-perfect identical boxes

✔ Good practice

IoU ≥ 0.5 is the community default for “correct enough.” Stricter tasks demand 0.75.

An IoU gallery



- Left: separate objects. Middle: partial overlap — likely two things. Right: near-perfect box.
- Green shows the intersection; IoU grows as the two boxes line up.

Key idea

Train IoU-reading by eye: it is the number you will debug detectors with most.

NMS: keeping the best box

Non-Maximum Suppression (NMS)

The greedy recipe, per class:

1. Sort all boxes by score, high $\rightarrow$ low.
2. Take the top box; *keep* it.
3. Remove every remaining box whose IoU with it exceeds a threshold τ (e.g. 0.5).
4. Repeat with the next surviving box until none remain.

Why "non-maximum"?

For each cluster of overlapping boxes we keep only the *maximum*-scoring one and *suppress* the rest. Duplicates gone; distinct objects kept.

NMS, by hand

Three boxes, threshold $\tau = 0.5$:

Box	Score	IoU with kept	Action
B1	0.90	—	**keep** (highest)
B2	0.80	0.70 vs B1	suppress ($> \tau$)
B3	0.75	0.10 vs B1	**keep** (different object)

- Start with B1 (top score). B2 overlaps it heavily $\Rightarrow$ drop as a duplicate.
- B3 barely overlaps $\Rightarrow$ it is a *second* object; keep it.
- Output: $\{$B1, B3$\}$. Three noisy boxes became two clean detections.

Choosing the NMS threshold

✗ τ too low (0.2)

Aggressive: two *real* objects that stand close (two dials) get merged into one — missed detections.

✗ τ too high (0.9)

Timid: duplicate boxes survive — one meter reported three times.

Key idea

$\tau \approx 0.5$ is the usual compromise. Crowded scenes need care — or "Soft-NMS," which *lowers* scores instead of hard-dropping.

In-class exercise

In-class exercise (10 min)

Compute overlap and clean up

1. Box $P = (0, 0, 4, 4)$ and $Q = (2, 2, 6, 6)$. Compute the intersection area, union area, and IoU.
2. Two boxes do not overlap at all. What is their IoU, and why can't intersection be negative?
3. You have boxes with scores $0.95, 0.9, 0.6$; the first two have IoU 0.8, the third has IoU 0.05 with both. Run NMS at $\tau = 0.5$: what survives?
4. A 600×400 image, stride 10, 4 scales. Roughly how many sliding windows?

Debrief

- (1) inter $= 2 \times 2 = 4$; areas 16 and 16; union $= 16 + 16 - 4 = 28$; IoU $= 4/28 \approx 0.14$.
- (2) IoU $= 0$. Overlap width/height are *clamped* at 0 — non-touching ranges give 0, never a negative area.
- (3) Keep 0.95; suppress 0.9 (IoU $0.8 > \tau$); keep 0.6 (IoU 0.05). Survivors: $\{0.95, 0.6\}$.
- (4) $\frac{600 \cdot 400}{10^2} \times 4 = 2400 \times 4 = 9600$ windows — and that's just one aspect ratio.

Key idea

You can now score a box against the truth and clean a pile of boxes into final detections.

Wrap-up

Common pitfalls

✗ Watch out

- Forgetting to *clamp* the overlap: negative width/height must become 0, not a phantom area.
- Double-counting in the union — subtract the intersection exactly once.
- Running NMS *across* classes: suppress within a class, or a meter kills a nearby dial.
- Treating a low-IoU neighbour as a duplicate — it is often a second, real object.

Good habits

✔ Good practice

- Decide your box format ($x_1 y_1 x_2 y_2$ vs centre) *once* and convert at the edges.
- Sanity-check IoU on two identical boxes ($= 1$) and two disjoint boxes ($= 0$).
- Keep NMS per class, and log how many boxes it removes — a great debugging signal.

What you can already do

- State the difference between classification, localization, and detection.
- Describe a box as 4 numbers and localization as regression on those numbers.
- Explain the sliding-window sweep and why the image pyramid handles scale.
- Compute IoU by hand and run NMS on a small set of boxes.

Next session

Session 2 — from brute force to a real detector

- Why sliding windows are too slow, and how **region proposals** trimmed the search.
- The leap to **single-shot** detectors (**YOLO**): one grid, one forward pass.
- **Anchor boxes**, the objectness/confidence score, and the output tensor.
- Measuring a detector honestly with **precision/recall** and **mAP** — then the DocScan deliverable.

Bring today's IoU and NMS firmly in mind — every modern detector leans on both.

Detection = “what” plus “where.”

Today: boxes, the sliding-window sweep, IoU to score overlap, NMS to keep the best.
Next: make it fast.